

[Open in app](#) ↗[Data Science Colle...](#) · [Follow publication](#)

You're reading for free via [Anubhav's](#) Friend Link. [Upgrade](#) to access the best of Medium.

★ Member-only story

RAG Chunking That Works: Semantic Splitting, Overlap, and Eval-Driven Tuning

Most RAG failures start with bad chunks. Here's how to split technical docs without wrecking retrieval quality.

14 min read · Apr 14, 2026



Anubhav

Follow

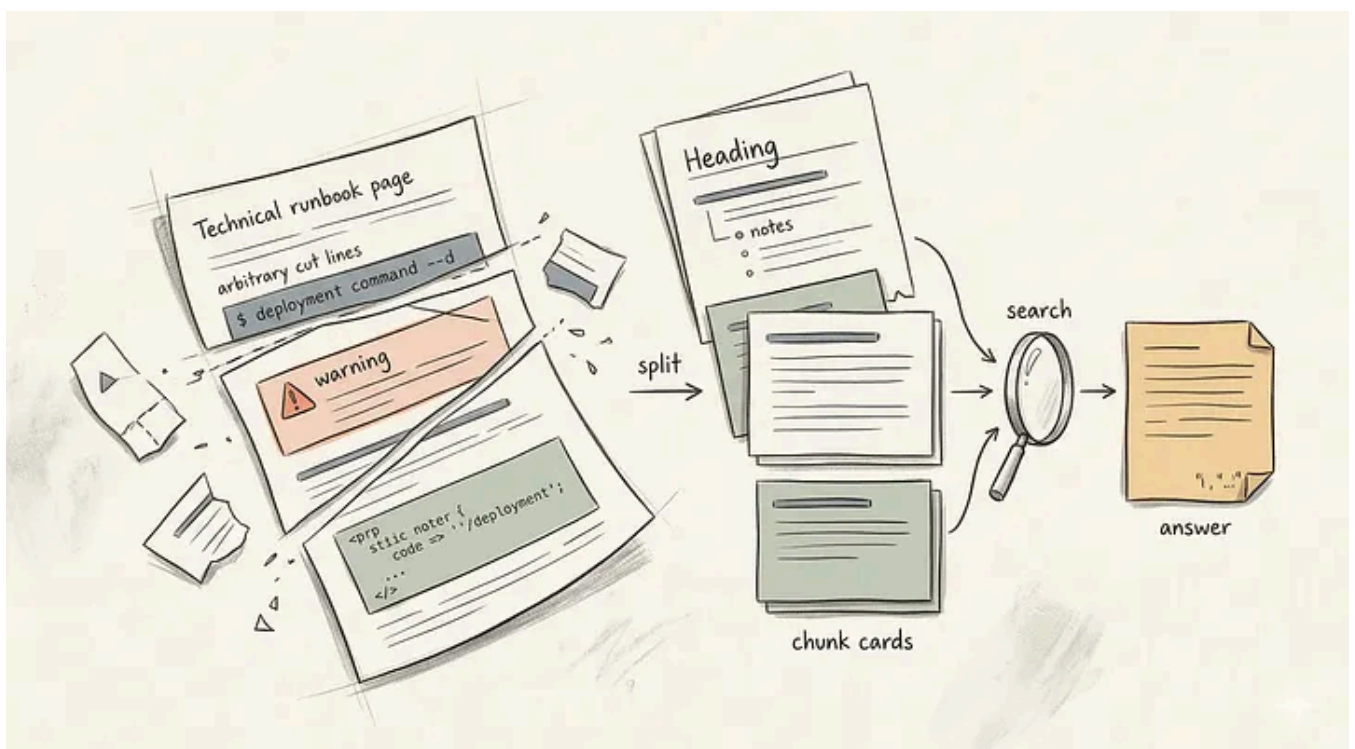


Listen



Share

... More



Teams spend weeks debating embedding models, vector databases, and prompt design. Then they split a critical runbook into arbitrary 400-token slices and wonder why the retrieval feels bad. The model is not the first thing failing in these systems. The chunk boundary is.

In the last post we mapped the modern RAG stack end to end. Now we zoom into one subsystem that quietly shapes everything downstream.

Not a medium member? Read the complete article [here](#).

Picture a standard internal docs copilot for an engineering team. The right answer may already be in the corpus, yet the assistant still returns something incomplete. You can tweak the prompt, swap the embedding model, and still get nowhere. But if you look at the actual text the database is returning, the problem becomes obvious.

The deploy procedure was split across two chunks, the warning was separated from the command it applied to, and a code block was retrieved without the explanation that made it safe to use. The system found text that looked related, but not enough to...

Chunking is not just a preprocessing step, it is a part of your retrieval quality. Your retriever does not search documents. It searches chunks. If you split your documents badly, the model never even sees a coherent answer.

The core idea: retrieval works on chunks, not documents

Most people think their RAG system searches docs. They imagine the database scanning entire markdown files to find the right page. What actually happens is entirely different.

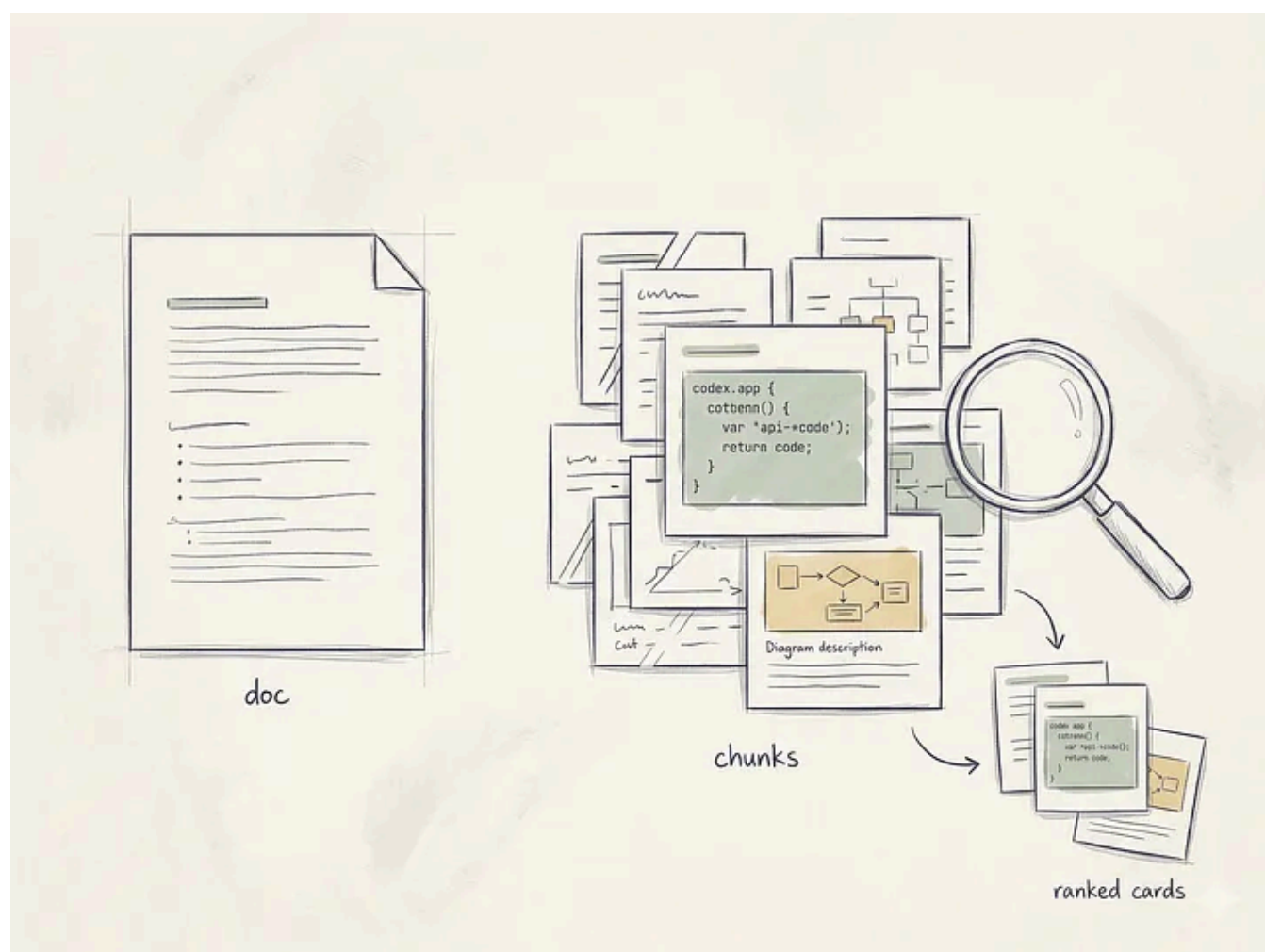
The index contains chunks. The retriever scores chunks. The reranker reranks chunks. The prompt gets chunks. The model answers from chunks.

The vector database only knows about the specific string of text you gave it. It has no concept of the paragraphs that came before or after it unless you explicitly program that relationship. If the original document was perfectly coherent but the chunk is an incoherent fragment, retrieval quality is already compromised. A bad chunk turns a good document into bad evidence.

Before we compare strategies we need to define what a good chunk actually is. A good chunk should preserve one coherent concept. It might be one procedural step

sequence. It might be one configuration explanation. It should definitely keep a warning attached to the step it modifies. It should keep a code example with the explanation that makes it usable.

A good chunk should not merge unrelated topics just to fill a token quota. It shouldn't split procedures in half. It shouldn't separate commands from warnings or detach examples from context. It shouldn't create fragments that require the missing neighboring chunk to make sense. Chunking is not just about size. It is about preserving meaning at retrieval granularity.



Major chunking strategies

There are a few distinct ways to cut up text. They range from extremely simple to highly complex. We will look at the mechanics of each and see where they break down.

Fixed-size chunking

This is the most basic approach. You split the text every N characters or tokens. You take a string, count to 500, make a cut, and repeat.

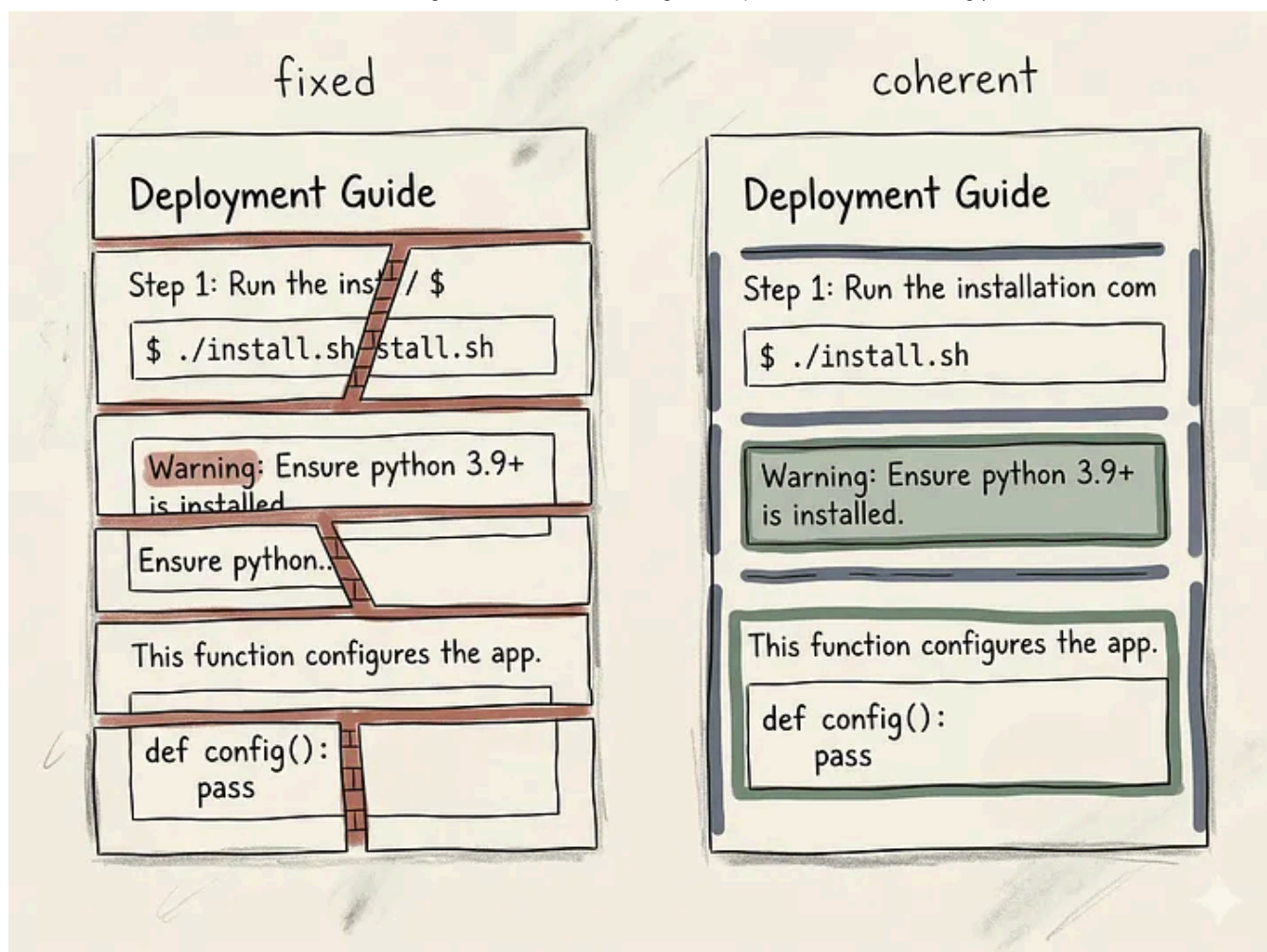
People use it because it is easy and fast. It is built into almost every framework by default. It is sometimes acceptable for uniform prose corpora like novels or long essays where the exact boundary doesn't ruin the meaning. It serves as a low-effort baseline when you are just trying to get a prototype running.

Where it fails completely is technical docs. It breaks code-heavy docs. It ruins documents with lists or tables.

If you use fixed-size chunking on a deployment guide, you will inevitably end up with a command in one chunk and the caveat in the next. A troubleshooting flow will get cut mid-step. An API endpoint description will be split right between the parameters and the JSON response example.

```
def chunk_fixed(text: str, size: int = 400, overlap: int = 50) -> list[str]:
    words = text.split()
    chunks = []
    i = 0
    while i < len(words):
        chunks.append(" ".join(words[i : i + size]))
        i += size - overlap
    return chunks
```

This code is fast. It is also completely blind to the structure of the text it is cutting. It just counts spaces and slices arrays. If word number 400 happens to be right in the middle of a critical database password string, it cuts the string in half. The LLM gets half a password and hallucinates the rest.



Recursive chunking

Recursive chunking tries to be a bit smarter. Instead of just counting words, it splits by larger semantic units first. It looks for headings. If the section under a heading is too big, it splits by paragraphs. If a paragraph is too big, it splits by sentences. It falls back down this list of separators until the resulting chunk fits your size constraints.

This helps because it preserves more natural boundaries than fixed-size splitting. A paragraph usually contains a complete thought. Keeping paragraphs intact makes the chunks much more readable for the LLM.

Where it still fails is that structural markers like headings may not capture the full semantic unit. Docs can be structurally messy. People write markdown with inconsistent heading levels. Important linked content might still be separated if a section is unusually long.

It is a better default than fixed-size chunking. It serves as a good baseline when you do not yet have full structure-aware parsing built for your specific document types.

```

import re

def chunk_recursive(text: str, max_tokens: int = 512) -> list[str]:
    separators = [
        r"\n#{1,6}\s",    # headings
        r"\n\n",          # paragraphs
        r"(?<=\.)\s",     # sentences
    ]
    return _split_recursive(text, separators, max_tokens)

def _split_recursive(text: str, seps: list[str], max_t: int) -> list[str]:
    if len(text.split()) <= max_t or not seps:
        return [text.strip()] if text.strip() else []

    parts = re.split(seps[0], text)
    chunks = []
    for p in parts:
        if len(p.split()) <= max_t:
            chunks.append(p.strip())
        else:
            chunks.extend(_split_recursive(p, seps[1:], max_t))
    return [c for c in chunks if c]

```

This logic tries to respect the author's original paragraph breaks. It uses regular expressions to find the natural pauses in the text. But it still doesn't truly understand what the text means.

Semantic chunking

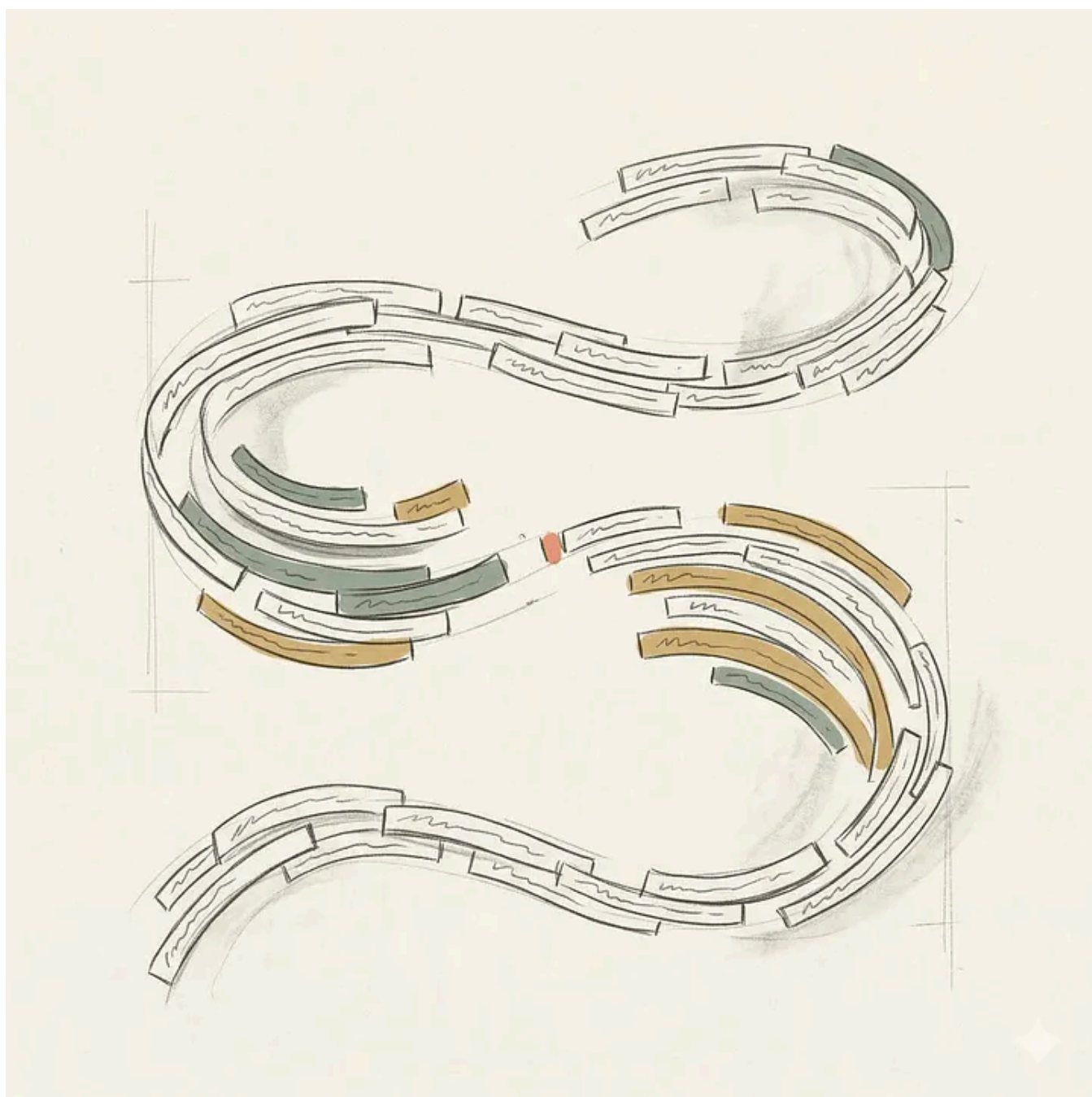
Semantic chunking takes a different approach. It splits based on shifts in topic or meaning rather than raw size or punctuation.

The typical implementation works by splitting the text into individual sentences. It then calculates an embedding vector for every single sentence. It compares the cosine similarity of sequential sentences. When the similarity drops below a certain threshold, it assumes the topic has changed and creates a chunk boundary right there.

This helps because it preserves coherent meaning better. It reduces arbitrary fragmentation. If an author writes a big wall of text about one specific database migration, semantic chunking will try to keep it together. When the author shifts to talking about cache invalidation, the chunker makes a cut.

The tradeoffs are significant. It is much more expensive to run because you have to embed every single sentence in your entire corpus just to figure out where to make the cuts. If you have a knowledge base with a million sentences, you are making a million API calls to your embedding provider before you even index the chunks. Therefore, it is much more complex to build. It can be hard to reason about without evaluation because the boundaries are determined by a black-box embedding model rather than explicit rules.

The best use case is knowledge-heavy corpora where topic boundaries matter more than document layout alone. Think of legal contracts or long-form research reports where the formatting is just plain text but the ideas shift frequently.



Structure-aware chunking

Structure-aware chunking uses document structure explicitly. It parses headings, subheadings, numbered procedures, code blocks, tables, callouts, and warnings.

It is the best fit for technical documentation. It preserves the exact units users actually need when they ask a technical question. A variant of this is exactly what you need to process a GitHub wiki or an internal developer portal.

If you have a markdown file, a structure-aware chunker knows that a block starting with `> **Warning**` is critically tied to the paragraph immediately preceding it. It knows that a code block enclosed in triple backticks should never be split down the middle.

The tradeoffs are that it depends heavily on parser quality. You have to write custom implementation work for different formats. A structure-aware chunker for Markdown looks very different from one for HTML or PDF.

```

HEADING = re.compile(r"^(#{1,6})\s+(.+)$", re.MULTILINE)
_CODE = re.compile(r"```[\s\S]*?```")
_WARNING = re.compile(r">\s*\*\*(Warning|Note|Caution)\*\*.*", re.MULTILINE)

from dataclasses import dataclass

@dataclass
class Chunk:
    content: str
    heading_path: list[str]
    doc_id: str
    meta: dict
    has_code: bool = False
    has_table: bool = False

def chunk_structured(text: str, max_tokens: int = 512) -> list[Chunk]:
    parts = _HEADING.split(text)
    chunks = []
    stack = []

    if parts[0].strip():
        chunks.append(Chunk(
            content=parts[0].strip(), heading_path=[], doc_id="",
            meta={}, has_code=bool(_CODE.search(parts[0])),
        ))

    for i in range(1, len(parts) - 1, 3):
        level = len(parts[i])
        title = parts[i + 1].strip()
        body = parts[i + 2].strip() if i + 2 < len(parts) else ""

```



```

    if not body:
        continue

    while stack and stack[-1][0] >= level:
        stack.pop()
    stack.append((level, title))
    path = [h[1] for h in stack]

    sections = _split_preserving_warnings(body, max_tokens)
    for sec in sections:
        chunks.append Chunk(
            content=sec, heading_path=path, doc_id="", meta={},
            has_code=bool(_CODE.search(sec)),
            has_table=bool("|" in sec and "---" in sec),
        )
    return chunks

def _split_preserving_warnings(text: str, max_t: int) -> list[str]:
    paras = [p.strip() for p in text.split("\n\n") if p.strip()]
    merged = []
    i = 0
    while i < len(paras):
        block = paras[i]
        while i + 1 < len(paras) and _WARNING.match(paras[i + 1]):
            block += "\n\n" + paras[i + 1]
            i += 1
        merged.append(block)
        i += 1

    result = []
    buf = []
    buf_t = 0
    for m in merged:
        t = len(m.split())
        if buf and buf_t + t > max_t:
            result.append("\n\n".join(buf))
            buf = []
            buf_t = 0
        buf.append(m)
        buf_t += t
    if buf:
        result.append("\n\n".join(buf))
    return result

```

This code tracks the heading path. If it extracts a paragraph deep inside a document, it remembers that this paragraph belongs under “Deploy Guide -> Prerequisites -> Network Config”. We will see why that matters when we talk about metadata. It also actively looks for warnings and glues them to the preceding text block.

Overlap: when it helps and when it hurts

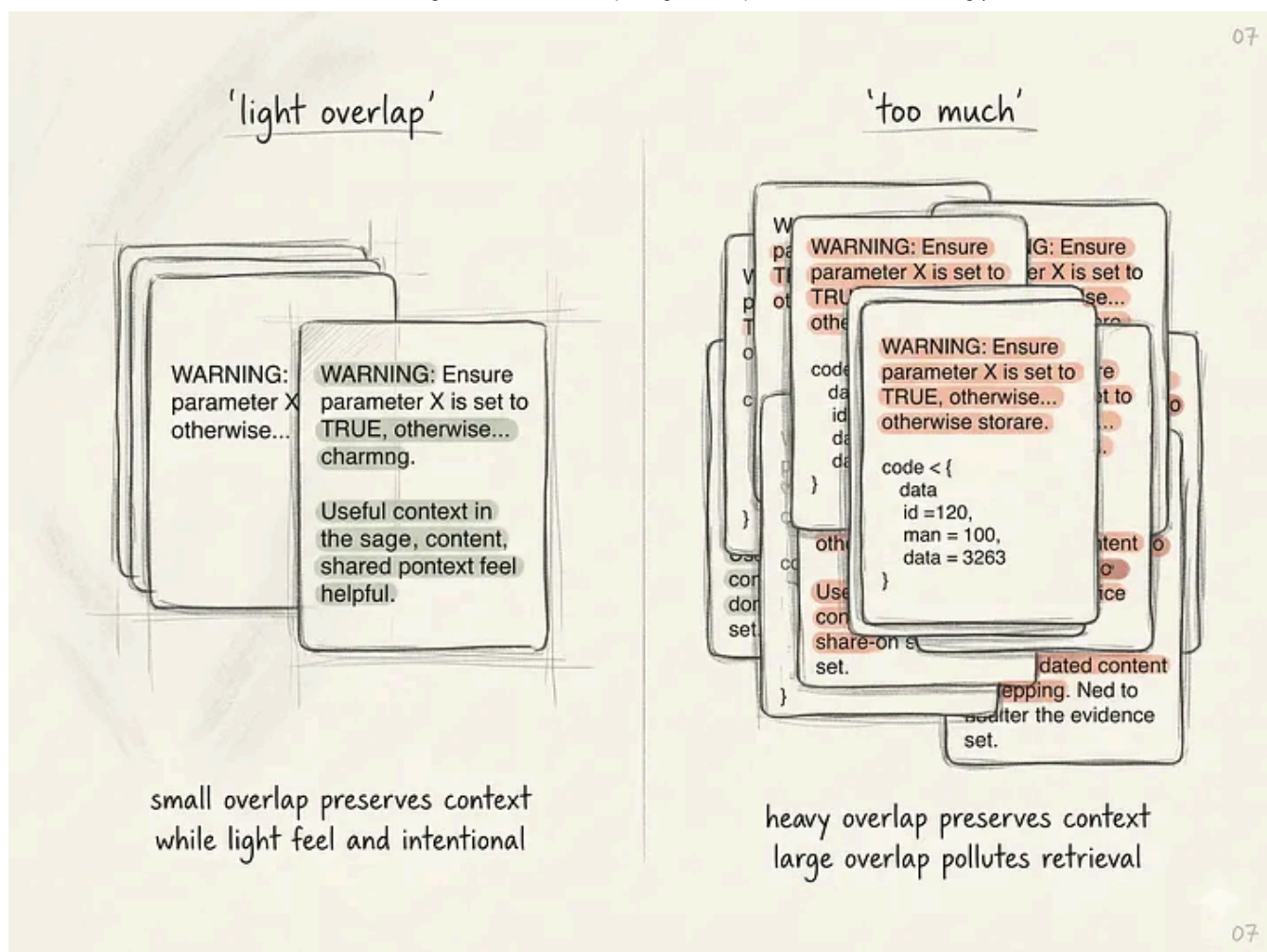
Overlap is one of the most misused knobs in RAG. Most tutorials tell you to set a chunk size of 500 and an overlap of 50. People copy that setting and never think about it again.

Overlap exists to preserve context near chunk boundaries. If a sentence gets cut in half, or if a pronoun in the next chunk refers to a noun in the previous chunk, overlap gives the embedding model enough context to understand what is going on. It acts as a sliding window over the text.

It helps for long procedures that simply cannot fit in one chunk. It is useful for code and explanation pairs. It helps with version-sensitive instructions where neighboring context matters.

But overlap hurts just as often. It duplicates near-identical chunks in your top-K results. If you have a large overlap and a user searches for a specific term, the vector database might return three chunks that are 80% identical because they overlap heavily. This pollutes retrieval with repetitive evidence. It wastes your context budget and makes citations noisy and repetitive for the user.

Overlap is not free context. It is a strict recall and precision tradeoff. Massive overlap is usually a band-aid for bad chunking. If you split your text intelligently at semantic boundaries, you need very little overlap.



Metadata and chunking belong together

Each chunk should carry metadata. It needs the source document name and heading or section path we extracted earlier. It needs the doc type, the service name, the version, the updated timestamp, and the owner or team.

This matters because retrieval is significantly better with filtering. If a user asks “how do I deploy the payment service”, you don’t just want to rely on vector similarity. You want to filter the vector database for `service_name == "payment"` before running the cosine similarity search. Most modern vector databases like Pinecone or Qdrant allow you to pass metadata filters alongside your query vector. This narrows the search space and eliminates irrelevant hits.

Reranking is also better with context. If the reranker model sees the heading path attached to the text, it understands the context of a generic paragraph. Citations become interpretable for the end user. Stale content is easier to detect and filter out.

A chunk without metadata is just floating text. It has no anchor in reality. When you pass a chunk to an LLM, you shouldn’t just pass the raw text. You should format it like this:

Source: payment-api-runbook.md

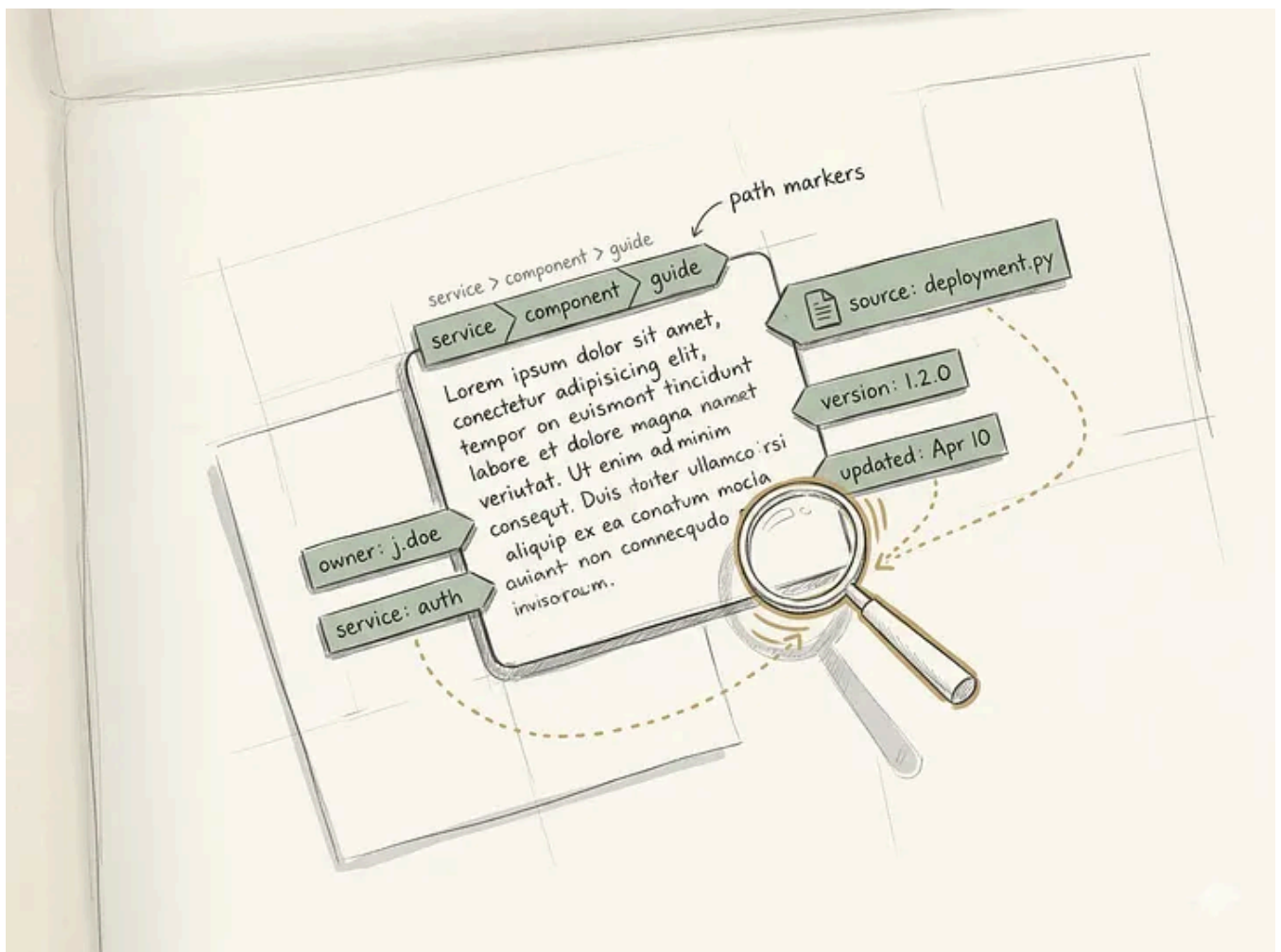
Section: Deploy Guide > Prerequisites

Last Updated: 2026-02-15

Ensure kubectl access and DEPLOY_TOKEN is **set**.

> ****Warning**** Without DEPLOY_TOKEN the deploy will silently fail.

This gives the LLM the context it needs to write a grounded, accurate answer. It also gives the LLM the exact string it needs to write a proper citation at the end of its response.



How chunking failures show up in production

When you build these systems, you start to recognize the symptoms of bad chunking. They usually act as model failures.

The most common is the “kind of right” answer. The retrieved chunk is topically related but not operationally sufficient. The user asks how to configure a timeout.

The LLM explains what the timeout does, but misses the exact YAML syntax because it was in the next chunk.

Then you have the missing final step. The answer explains most of a procedure but omits the one crucial command or warning at the very end. The user follows the instructions and breaks their environment.

The stale answer problem happens when a chunk from an older doc version outranks the current one. Vector similarity doesn't understand time. An old doc explaining how to configure a legacy system might have higher keyword overlap with the user's query than the new, streamlined doc.

You also see citations that look real but are useless. The answer cites a chunk that only partially supports the claim. The user clicks the citation link, reads the paragraph, and realizes it doesn't actually contain the exact command the LLM provided. The LLM hallucinated the rest based on its training data because the chunk was incomplete.

The giant-clump failure happens when you set your chunk size too large. The chunk is so big that retrieval technically works, but prompt assembly gets diluted. The LLM gets distracted by 1000 tokens of irrelevant text surrounding the 50 tokens that actually matter, and answer quality drops.

The tiny-fragment failure is the exact opposite. The chunk is so small the model has to hallucinate the missing context. It sees a chunk that just says `docker-compose up -d` and invents a whole narrative around it.

Many "model failures" are actually chunk design failures. You cannot prompt your way out of broken evidence.

Eval-driven chunk tuning

This is where engineering discipline comes in. The anti-pattern in RAG development is choosing a chunk size once, adding overlap randomly, never inspecting retrieved evidence, and tuning by vibes. You change a setting, ask the bot a question, see if it looks okay, and push to prod.

The engineering approach is creating a small evaluation set and comparing chunking strategies systematically.

You need to evaluate retrieval hit quality. Did the top 3 chunks contain the actual answer? You need to evaluate answer completeness. Did the LLM have enough context to write a full response? You need to check citation usefulness and chunk coherence. You need to see how the chunking strategy handles ambiguity and stale-doc sensitivity.

You don't need thousands of questions for this. You need a curated set of 50 to 100 questions that represent real user intent.

Suggested eval set categories include exact lookup questions like "What is the max timeout for the API?". Procedure-following questions like "How do I rollback a database migration?". Troubleshooting questions like "Why am I getting a 502 on the auth service?". Version-sensitive questions and configuration questions also matter.

You run this eval set against different chunking strategies. You compare fixed-size against recursive against structure-aware. You test different chunk sizes and different overlap levels. You test with and without metadata-aware filters.

```
from dataclasses import dataclass

@dataclass
class EvalCase:
    question: str
    expected_chunk_contains: str
    category: str

def eval_chunking(
    text: str,
    cases: list[EvalCase],
    strategies: dict[str, callable],
    embed_fn: callable,
    search_fn: callable,
    top_k: int = 3,
) -> dict[str, dict]:
    results = {}

    for name, chunk_fn in strategies.items():
        chunks = chunk_fn(text)
        contents = [c.content if hasattr(c, 'content') else c for c in chunks]
        chunk_embs = embed_fn(contents)

        hits = 0
        total = len(cases)
        for case in cases:
            q_emb = embed_fn([case.question])[0]
```

```

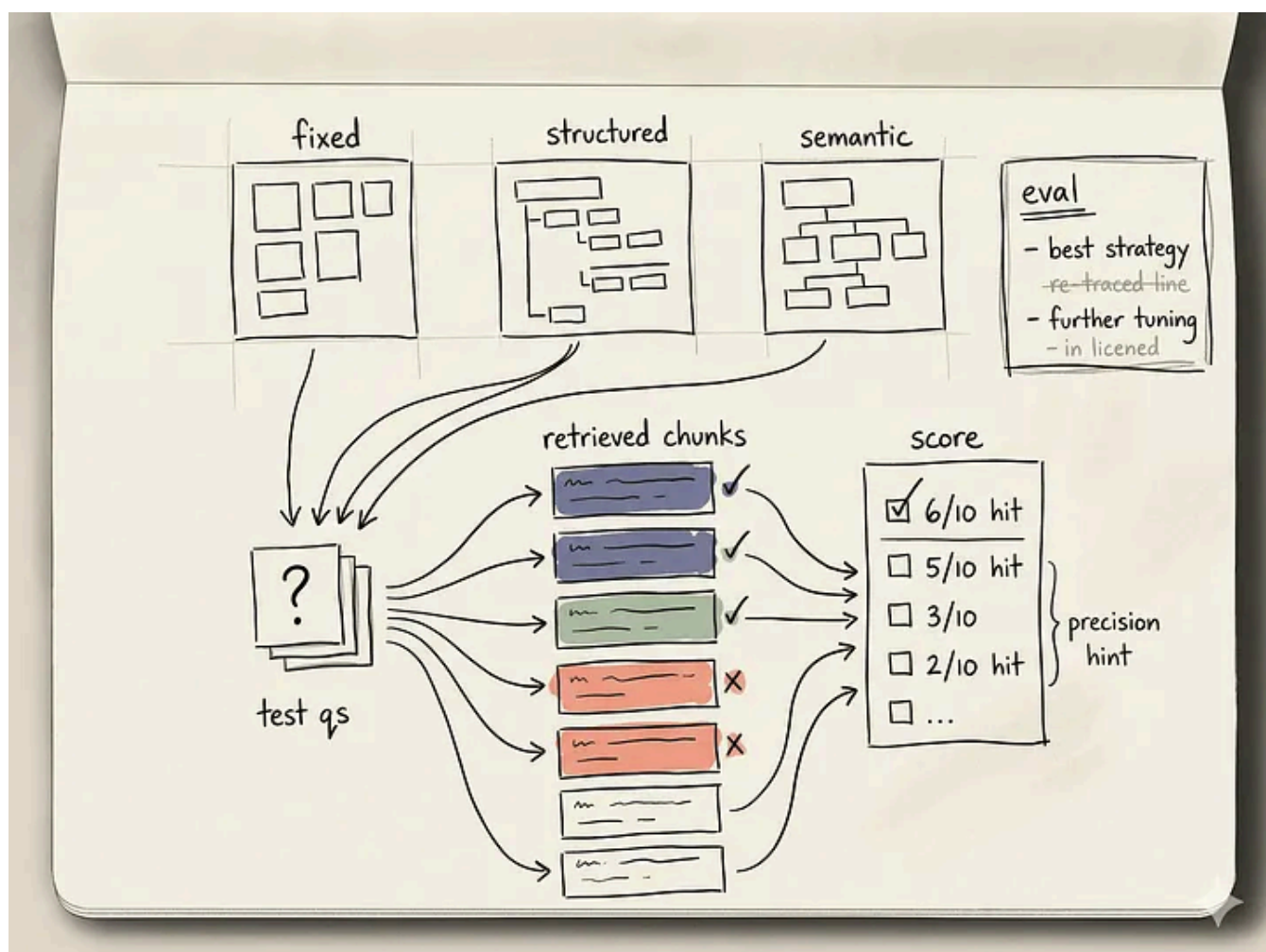
idxs = search_fn(q_emb, chunk_embs, top_k)
retrieved = [contents[i] for i in idxs]

if any(case.expected_chunk_contains in r for r in retrieved):
    hits += 1

results[name] = {
    "hit_rate": hits / total if total else 0,
    "num_chunks": len(chunks),
    "avg_tokens": sum(len(c.split()) for c in contents) // max(len(cont
}
return results

```

This approach lets you prove which strategy works. The `search_fn` simulates your vector database. The `top_k` parameter mimics how many chunks you plan to feed into your LLM prompt. You might find that structure-aware chunking increases your hit rate from 60% to 85% on procedural questions. Chunking should be tuned empirically. It should not be inherited from random blog defaults.



A practical default strategy

If you are building a system today and want a strong starting point, here is a recommended default for technical docs.

Start with structure-aware recursive chunking. Write a parser that respects your specific markdown or HTML flavor. Preserve headings, code blocks, warnings, and procedures as atomic units.

Use light overlap. Do not use huge overlap. If your chunks are semantically coherent, you only need a tiny bit of overlap to catch edge cases.

Attach rich metadata to every chunk. Inject the heading path directly into the text payload so the embedding model sees it.

Evaluate this baseline before you change embedding models or vector databases. It is strong enough for real engineering docs and simple enough to implement without building a massive research project.

An important nuance is that different document types may still need different chunking rules. One universal chunking policy across the whole corpus is often a mistake. A Slack message dump needs different splitting logic than a formal API specification.

Artifact pack

To make this actionable, here is a Chunking Tuning Checklist you can run through before you deploy your next pipeline.

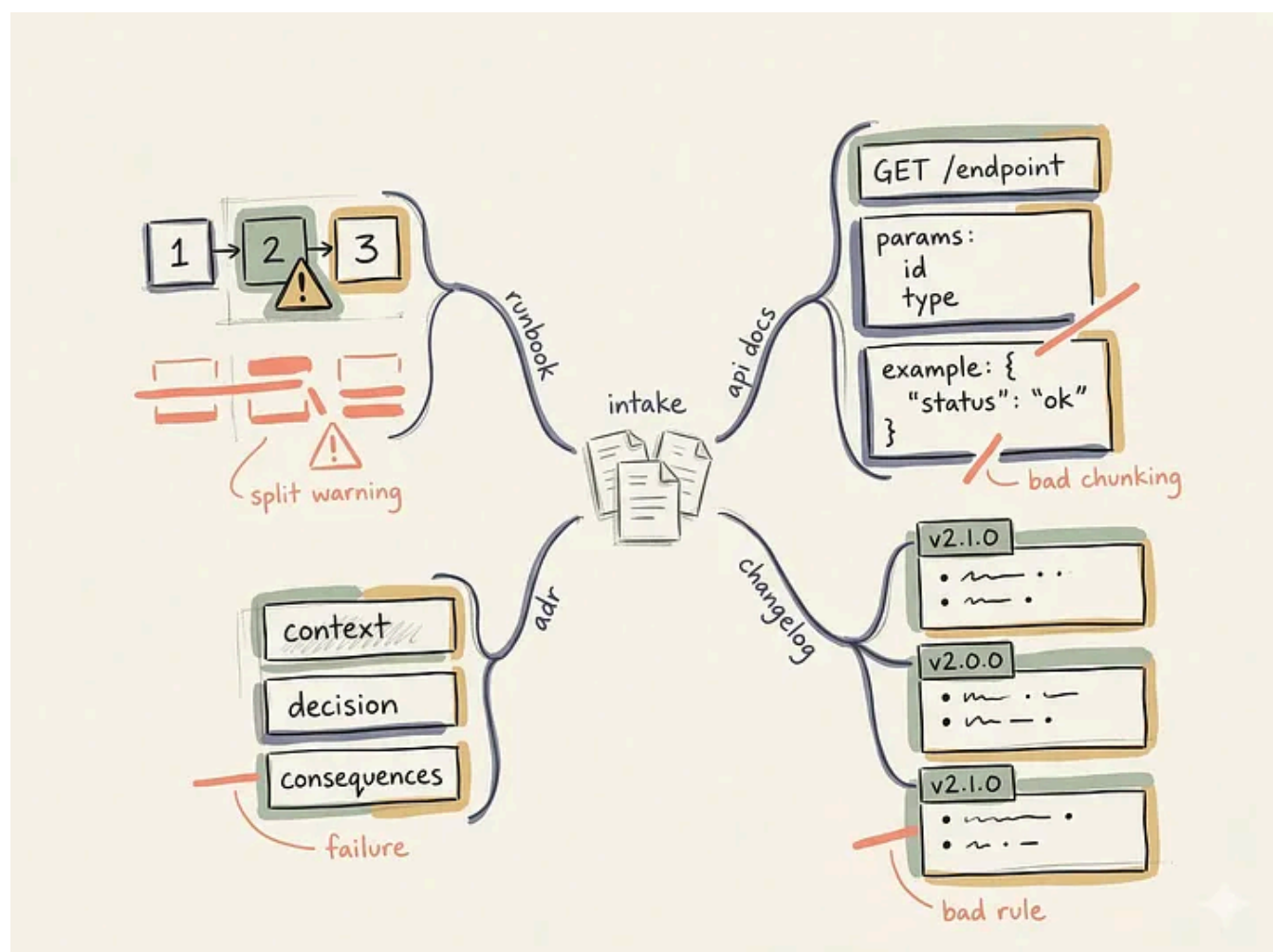
- Are we preserving semantic boundaries?
- Are warnings kept with the commands they modify?
- Are code blocks kept with their explanation?
- Does each chunk carry useful metadata (source, heading path, version)?
- Are we using overlap intentionally rather than blindly?
- Have we evaluated our chunking strategy on real questions?
- Are we inspecting the actual retrieved text during debugging, not just the final LLM output?

The code snippets provided throughout this post form the basis of a solid chunking pipeline. You can take the structure-aware chunker, adapt the regex to your specific documentation format, and run it through the eval harness to prove it works.

Conclusion

The chunk is the fundamental unit of evidence your system works with. If that unit is incoherent, retrieval quality collapses quietly. Good chunking makes the rest of the RAG pipeline easier. Bad chunking forces every later component to compensate for broken evidence.

Before you swap models, before you upgrade your vector database, and definitely before you start building complex multi-agent loops, look at your data. Make sure your chunks actually mean something.



What's Next

We have fixed the foundation. Our documents are now coherent, retrievable units of evidence. But finding the right chunks requires more than just vector similarity. In the next post, we will look at Hybrid Search. We will break down exactly when BM25

keyword search beats vector search, when vectors win, and how to combine them properly without destroying your latency.

Artificial Intelligence

AI Agent

Machine Learning

Programming

Software Engineering



Follow

Published in Data Science Collective

921K followers · Last published 10 hours ago

Advice, insights, and ideas from the Medium data science community



Follow

Written by Anubhav

796 followers · 28 following

I write deep-dives on agents, RAG, evals, and what actually breaks when you ship AI systems

